

DCNNBiLSTM: An Efficient Hybrid Deep Learning-Based Intrusion Detection System

Vanlalruata Hnamte*, Jamal Hussain

Department of Mathematics and Computer Science, Mizoram University, Tanhril, Aizawl, Mizoram 796004, India

ARTICLE INFO

Keywords:

Deep learning
DDoS
IDS
Bilstm
CNN
DCNNBiLstm

ABSTRACT

In recent years, all real-world processes have been shifted to the cyber environment practically, and computers communicate with one another over the Internet. As a result, there is a rising number of network security vulnerabilities, and network administrators are unable to secure their networks from all forms of cyberattacks. Many techniques for network intrusion detection have also been developed. However, they encounter significant challenges as a result of the ongoing emergence of new vulnerabilities that present systems cannot comprehend. We are motivated by deep learnings exceptional performance in various detection and identification tasks, we present an intelligent and efficient network intrusion detection system (NIDS) based on Deep Learning (DL). In this study, we present a deep learning-based IDS for attack detection. The model has been trained with real-time traffic datasets; CICIDS2018 and Edge_IoT datasets. The performance of the model is investigated using multi-class classification and achieved a 100% and 99.64% accuracy rate respectively when trained and tested with the datasets.

1. Introduction

In an increasingly digital and diverse environment, it is extremely difficult to protect private information using traditional security solutions. Furthermore, the number of cyber-attacks on computer networks has increased in recent years. Various approaches for identifying suspicious network activity with intrusive behaviors including obscurity, variety, entanglement, and changing tendency are constantly emerging. With the implementation of artificial intelligence-based algorithms, the accuracy of the intrusion detection and prevention system has lately been greatly enhanced.

The IDS is a mechanism that monitors network traffic to detect suspicious or abnormal activity and then takes preventative measures against intrusion threats. As a function, intrusion detection systems are classified into two types: (1) NIDS and (2) HIDS. The NIDS is often implemented or positioned at crucial network points to guarantee that it covers traffic that is more vulnerable to attack, whereas the HIDS systems function on any device on the network that has an Internet connection. There are two basic strategies for detecting intrusion, namely (1) IDS based on anomalies and (2) IDS based on signatures [1]. Signature-based intrusion detection focuses on recognizing æsignatureg patterns of intrusion occurrence and is perfect by updating the signature database to the latest trend or zero-day attack pattern regularly. The anomaly-based intrusion detection system (IDS) (i.e., Behavior-based Detection) compares

trustworthy behavioral patterns with novel behaviors based on frequent activity monitoring. When an administrator gets an alert from the IDS system, it employs Intrusion Prevention Systems (IPS) to prevent threats such as Trojan horses, DDoS attacks, and so on [2].

Implementing an effective network intrusion detection system is one of the critical issues that must be addressed in network security (NIDS). Despite significant developments in NIDS, the majority of existing NIDS systems are focused on signature-based approaches rather than anomaly detection techniques [3]. There are several reasons for not adopting anomaly detection techniques, including the systems behavioral dynamics, the durability of training data, the collection of accurate training data, high costs, and error rates owing to the dynamic nature of the data. Current procedures may result in erroneous and ineffective NIDS verification techniques and solutions. We require an effective intrusion detection system to address the inadequacies of todays most advanced networks.

The development of the intrusion detection system spans more than three decades, and it has progressively evolved into a marketable product. And it has become the primary subject and direction of Internet security research [4]. Technology for intrusion detection is continually growing and being updated. Intrusion detection systems have consistently incorporated technologies like data mining, pattern recognition, fuzzy set theory, and neural networks. Develop intrusion detection to a new level by enhancing and enhancing the effectiveness of intrusion

Abbreviations: CNN, Convolutional Neural Network; IDS, Intrusion Detection System; DCNNBiLstm, Deep Convolutional Neural Network Bidirectional-Long-Short Term Memory; Bilstm, Bidirectional-Long-Short Term Memory; DDoS, Distributed Denial of Service.

* Corresponding author.

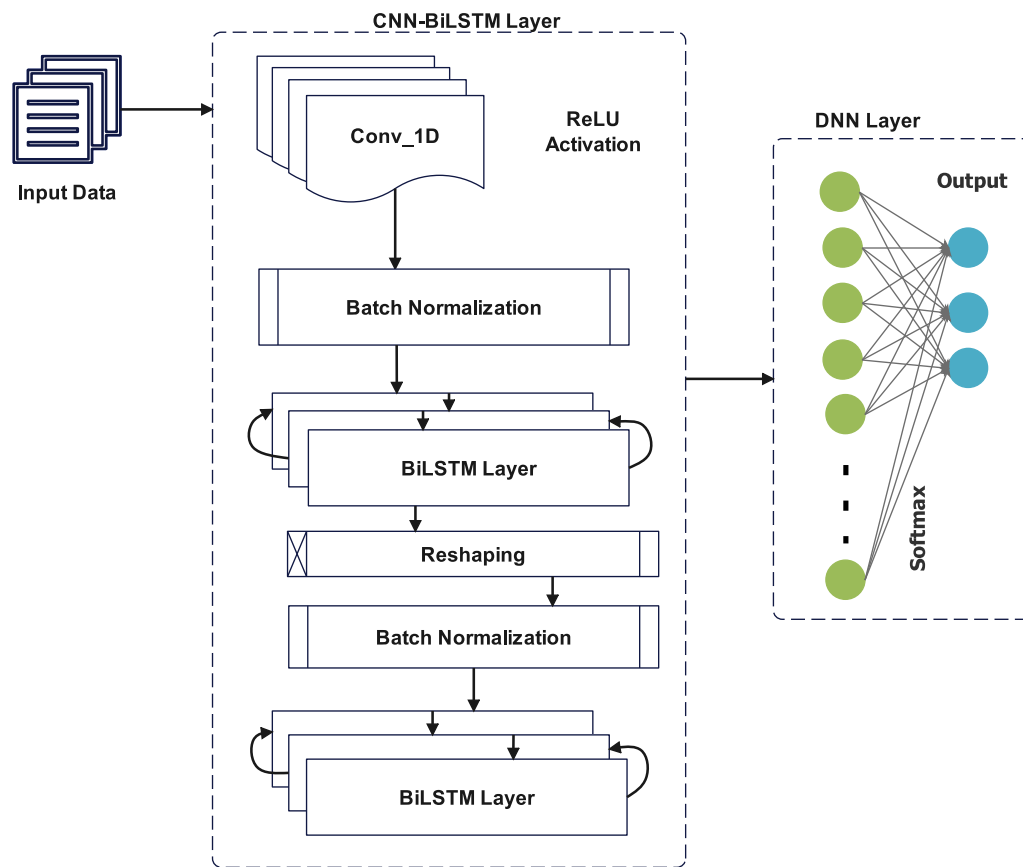


Fig. 1. DCNNBiLSTM Approach.

detection on a consistent basis [5]. Deep learning models are often the most intelligent in the world of artificial intelligence. The deep neural network can automatically learn the latent representation necessary for intrusion detection from raw information. Furthermore, the deep neural networks generalization capacity prevents the IDS from overfitting by forming associations beyond the inputs immediate neighbors [6]. Specifically, the deep neural network is deemed capable of improved data modeling when handling a big dataset about the attack profile of shilling attackers as indicated by recommendation systems. Specifically, deep learning models are referred to as feature extractor networks. They are responsible for differentiating between the authentic user and attacker profiles. This deep learning models major purpose is to learn the most potentially helpful high-level and deep properties for categorization or target identification. Implementation of feature extractors may be based on the calculation of convolution between the data and certain filters, followed by the downsampling operation to preserve just the most relevant features.

Traditional intrusion detection solutions based on deep learning algorithms often use binary classifiers to classify attacks. Due to the influence of data dimensions on huge high-dimensional data, the training model often results in a poor attack detection rate, which reduces the overall detection efficiency. Deep learning may extract higher-dimensional characteristics from the original data, resulting in a more accurate classification model when used in the development of an intrusion detection system. The proposed model is depicted in Fig. 1. In order to increase the accuracy of network intrusion detection, this study presents the following innovative ideas:

- Create a model that combines CNN and BiLSTM networks with a more hidden layer. Perform data cleansing, and then incorporate the attention mechanism to enhance the classification accuracy of the model.

- By manually removing characteristics from the design feature and directly using the deep learning model to learn the characterization features of high-dimensional data, the detection accuracy of the method is enhanced.

The rest of this article is organized as follows: Section 1 is the introduction to our study. Section 2 summarizes related works. In Section 3, we discuss the datasets used for this study and preprocessing datasets. Section 4 presents the proposed model in detail. Section 5 illustrates the experimental study of the result and discussion. We evaluate the experimental results and make a comparative analysis with other intrusion detection methods. Finally, Section 6 concludes this article.

2. Related studies

Deep Learning models have lately played an important role in dealing with more complicated data representations. Because of its superior performance, the DL is receiving more attention and is being used in a variety of multidisciplinary study domains such as healthcare [7], vehicle design [8], manufacturing [9], and security [10,11].

Tian et al. [12] proposed an IDS utilizing a Radial Basis Function Network. By recognizing the distinguishing harmful features contained in the network, the neural network can quickly and efficiently recognize various infiltration actions. The computational findings demonstrated that the designed intrusion detection system is practical and effective at classifying attacks. The proposed radial basic function network work achieved 392 average iteration time, where the LLSDDoS¹ dataset was used for the training and testing. The dataset is outdated and does not reflect any modern attack scenario, and is very small in size.

¹ LLSDDoS https://archive.ll.mit.edu/ideval/data/2000/LLS_DDOS_2.0.2.html

Mohammad et al. [13] used a standard neural network to classify network threats in a system. A two-layer multi-layer perceptron was built using the backpropagation learning technique. The proposed method categorization accuracy could achieve a 90.78% accuracy rate. The study used KDDCUP99², ISCX³ and NSL-KDD⁴ datasets for training and testing the model. The dataset is outdated and does not reflect modern attack scenarios.

Diro and Chilamkurti [14] developed a deep learning-based distributed threat detection system for IoT networks, and compare the proposed model with the Shallow model. The authors suggested that this technology be deployed in the cloud system for attack detection and training purposes. In the performance evaluation, the publicly available dataset NSL-KDD was used and achieved a precision of 99.20% with binary classification, and 98.27% in multiclass classification, respectively.

Muna et al. [15] developed an anomaly detection system (ADS), to identify cyber threats in the industrial internet of things. The proposed system used DL techniques, namely the UDAE algorithm, to train the networks usual patterns of activity and create the appropriate settings. Publicly available datasets namely NSL-KDD and UNSW-NB15⁵ were employed in the performance evaluation, and the findings show a detection achieved 99% accuracy rate and a False Positive rate of 1.8%. Using the NSL-KDD for training, the model took an average time of 194.37 seconds, whereas, using UNSW-NB15 for training, the model took an average time of 227.30 seconds per epoch.

Vinayakumar et al. [16] provided a system for detecting botnet attacks termed C-CMU that employs a two-tier environment for monitoring DNS logs and searching the domain name created by the domain creation process using deep learning algorithms to decrease false alarm rates. The proposed model could achieve a 99.2% accuracy rate with test - I, 84.5% accuracy rate with test - II when trained with DS1-V1, and 73.1% accuracy rate with test - I, and 89.9% accuracy rate with test - II when trained with DS2-V2 dataset.

Parra et al. [17] suggested distributed architecture algorithms, namely a DCNN and an LSTM network, for detecting phishing and Botnet attacks and identifying an adversary who attempts to introduce irrelevant data. The Distributed CNN was employed in an IoT micro-security, whereas the LSTM model was employed at the backend server. The evaluation performance achieved results showing an accuracy rate of 94.30% for multiclass classification and 94.8% accuracy rate for binary classification throughout the evaluation phases, respectively. The N_BaIoT dataset was used with the proposed model on the paper [17].

Haddad Pajouh et al. [18] created an intrusion detection system for identifying IoT malware using an RNN. The experimental results showed that the 10-fold cross-validation study achieved the best accuracy rate of 98.18% and that it was more efficient than traditional ML classifiers such as NB, K-NN, RF, and DT.

Popoola et al. [19] created a hybrid technique termed LAE-BLSTM for detecting botnets. The proposed model was trained with BotIoT⁶ dataset. The proposed model distinguish attacks from normal traffic [19]. The evaluation result of the study showed that an accuracy rate of 91.89% with data size reduction was achieved.

Using the Deep Belief Network (DBN) approach, Manimurugan et al. [20] developed an IDS. The suggested method was tested using the publicly available dataset CICIDS-2017⁷ could achieve an accuracy rate of 99.37%.

NG and Selvakumar [21] provided a framework for anomaly identification, a vector convolutional deep learning (VCDL) approach. The proposed model was trained and tested using the BotIoT dataset. The accuracy rate is 99.7496% for multiclass classification.

The majority of the IDS reported in this section use outdated dataset and the proposed model were validated with mostly single dataset. The applicability of the proposed model should be validated with newer datasets as well as, small and large datasets.

2.1. CNN-Based IDS

CNN is a hierarchical model in the realm of network intrusion detection that excels in extracting local characteristics. A basic CNN model incorporates the three concepts of shared weights, spatial or temporal sampling, and local receptive fields. This provides a distinct benefit when dealing with statistically stable and locally relevant data. Fig. 2 depicts the principal block diagram of the CNN-based IDS.

Four phases comprise the workflow of a CNN-based network intrusion detection system.

1. Preprocess each record in the data collection for intrusion detection so that its data type format may be utilized as the CNN model's input.
2. The processed data are then fed to the CNN's input layer, where the convolution layer executes a convolution operation to properly extract the distinctive information of each record.
3. The pooling layer combines the feature points in the local neighborhood to produce new features. Pooling operations are intended to accelerate network training. The techniques of average pooling and maximal pooling are often utilized.
4. The last step involves the entry of data into the Softmax classifier through the fully connected layer in order to categorize the kind of intrusion.

The classification accuracy of CNN-based network intrusion detection systems is better than that of conventional approaches, and they can reliably extract local feature information. This technique cannot overcome the issue of long-term reliance since it lacks the ability to learn sequence correlation information. Its rate of precision must be enhanced.

2.2. LSTM-Based IDS

RNN is proficient at handling sequence data. However, there will be a gradient loss, gradient growth, and long-term reliance issues throughout the training process. The LSTM model's long- and short-term memory modules can alleviate the RNN's long-term reliance issue. As illustrated in Fig. 3, the LSTM module adds three gates and a cell state update to the RNN model's hidden layer.

The forget gate filters the state of the top layer of cells, retaining important information and erasing unnecessary information. Details explanation is available in the next section.

Motivated by the aforementioned results, this work develops a hybrid DL architecture for intrusion training prediction utilizing different datasets. Convolutional Neural Network with Bidirectional Long Short-term Memory (CNN-BiLSTM) is the model suggested in this paper. In addition, certain DL models were used as a comparison to highlight the superior efficiency of the proposed method. This paper proposes a hybrid model for a stable and high-performance strategy, as opposed to a number of previously completed studies that concentrated on constructing DL and ML models for intrusion detection. In contrast, big datasets and more current datasets are seldom employed in contemporary IDS research; hence, it is our major objective to use more recent datasets that represent modern attack scenarios in our study. In addition, the combination of the CNN and BiLSTM models is successful in forecasting network intrusion detection over the near run. CICIDS-2018 and Edge IoT were used to evaluate the hybrid architecture that was proposed. As a result, the study contributes to a deeper understanding of network intrusion detection and provides significant information for the future development of DL models for IDS.

² KDDCUP99 <https://www.kdd.org/kdd-cup/view/kdd-cup-1999/Data>

³ ISCX <https://www.unb.ca/cic/datasets/ids.html>

⁴ NSL-KDD <https://www.unb.ca/cic/datasets/nsl.html>

⁵ UNSW-NB15 <https://research.unsw.edu.au/projects/unswnb15-dataset>

⁶ BotIoT <https://research.unsw.edu.au/projects/bot-iot-dataset>

⁷ CICIDS-2017 <https://www.unb.ca/cic/datasets/ids-2017.html>

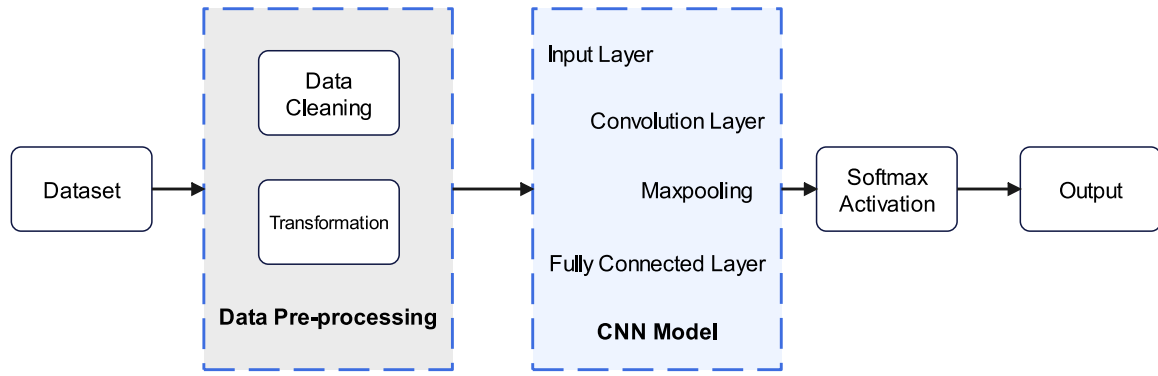


Fig. 2. CNN Model.

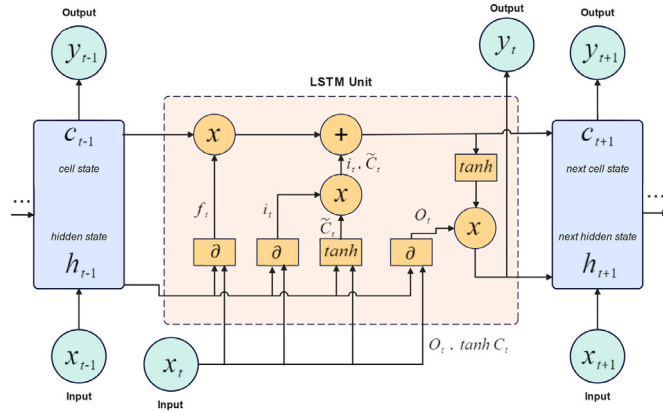


Fig. 3. LSTM Unit.

Table 1

Class Distribution of CICIDS-2018 DDoS-LOIC-UDP-HOIC.

Class	Instances	Percentage
DDoS attack-HOIC	686012	65.423%
Benign	360833	34.412%
DDoS attack-LOIC-UDP	1730	0.165%

mean is 0 and its variance is 1. After applying one hot encoding, we have 119 features; to standardize the feature space, we have used a standard scalar. Mathematically it can be represented as Eq. 1.

$$z = \frac{x - \mu}{sd} \quad (1)$$

Here, z represents the standard feature space of the x input data samples, μ is the mean, and sd is the standard deviation. The mathematical representation of the mean is $\mu = \frac{1}{N} \sum_{i=1}^N (x)_i$ and standard deviation is $sd = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2}$ here x_i is input sample.

3. Pre-processing

In order to construct a classification model for intrusion detection based on the combination of CNN and BiLSTM, the original dataset must be preprocessed utilizing techniques such as data cleansing and data transformation.

Missing, distorted, and incomplete data are referred to as "dirty data." The internal rules in the original data information are broken, leading to poor performance in data analysis and processing. Therefore, it is required to clean "dirty data" and transform it into data that satisfies the data quality standards. Missing values, erroneous data, misfits, and distortion are the primary obstacles to data cleansing. The solution is to utilize the same constant to fill in the missing data and to remove or replace the current special symbols and garbled codes.

3.1. One hot encoding

We used one hot encoding to transform categorical characteristics to numeric values since the majority of machine learning and deep learning models operate on numerical input. This approach counts the unique values for each characteristic and assigns a unique index to each value.

3.2. Data normalization

Depending on the DL model used, the data normalization method normalizes the feature values in the range $[1, +1]$ or $[0, +1]$. Data normalization, also known as standardization, reduces training time and accelerates model convergence. There are several strategies for normalizing data, including minmax scaling, standard scaling, and averaging. In this research, a standard scalar was used to normalize the data. Standard scalar used a standard normal distribution (SND), therefore it's

3.3. Dataset

The datasets are initially subjected to data cleaning in this study. Utilize CNN to extract parallel local features in order to acquire more complete local characteristics.

3.3.1. CICIDS2018

The CICIDS2018 (DDoS-loic-udp hoic 21-02-2018) dataset includes three kinds of attacks. We explore DDoS attacks, particularly against network intrusion detection. The CICIDS2018⁸ dataset was developed under the collaboration of the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC) [22]. The CICIDS2018 dataset comprises benign and up-to-date common threats, and it closely mimics the real-world data (PCAPs) gathered by the Canadian Institute for Cybersecurity. A DDoS attack aims to render a distributed network incapable of responding to user requests. The dataset comprises two forms of DDoS attacks, namely DDoS attack-HOIC (6,86,012 rows) and DDoS attack-LOIC-UDP (1730 rows). Whereas the rest (360,833 rows) belong to the benign class.

The dataset originally contains 80 features, but three features such as 'dst_port', 'protocol', and 'timestamp' were dropped since they don't play a significant role in our training and testing purposes. No null or empty data were found during the dataset analysis. The dataset is still very imbalanced and the details of the class distribution are shown in Table 1.

⁸ CICIDS2018 <https://www.unb.ca/cic/datasets/ids-2018.html>

Table 2
The total instances and types of records in the Edge_IIoT dataset.

IoT Traffic	Type	Instances	Percentage
Normal	Normal	1,615,643	72.80291%
Attack	DDoS_UDP	121,568	5.478008%
	DDoS_ICMP	116,436	5.246753%
	SQL_injection	51,203	2.307272%
	Password	50,153	2.259958%
	Vulnerability_scanner	50,110	2.25802%
	DDoS_TCP	50,062	2.255857%
	DDoS_HTTP	49,911	2.249053%
	Uploading	37,634	1.695836%
	Backdoor	24,862	1.120313%
	Port_Scanning	22,564	1.016762%
	XSS	15,915	0.71715%
	Ransomware	10,925	0.492294%
	MITM	1,214	0.054704%
	Fingerprinting	1,001	0.045106%

The dataset was divided into training and testing parts. The testing dataset consisted of 25% of the original dataset, while the training dataset comprised 75% of the original dataset.

3.3.2. Edge_IIoT

The Edge_IIoT [23] dataset is a new cybersecurity dataset for IoT and IoT applications, which was introduced by Ferrag et al. [23] for IoT environment as realistic cyber security dataset in 2022. It contains 62 features with 14 attack types. This dataset contains 14 distinct attacks against IoT and IIoT protocols, categorizing them into five distinct dangers: denial-of-service and distributed denial-of-service, information collecting, injection, man-in-the-middle, and malware. There is a significant relationship between 61 of the 1176 traits. Edge_IIoT [23] reports a total of 2,219,201 attacks, 1,615,643 of which are considered regular, and 603,558 are considered an attack. We used a stratification option to maintain the percentages consistent across all classes and used 20% of the sample for testing while keeping the remaining 80% for training. Table 2 shows the detailed distribution of the Edge_IIoT dataset instances.

From the Edge_IIoT dataset the features shown in Table 3 were dropped during the pre-processing stage.

Fig. 4 illustrated the distribution of the attack data from Edge_IIoT dataset. The data is an imbalance in terms of the normal class and the

Table 3
Features dropped from Edge_IIoT.

frame.time	ip.src_host	ip.dst_host
arp.src.proto_ipv4	arp.dst.proto_ipv4	http.request.full_uri
icmp.transmit_timestamp	http.request.uri.query	tcp.options
tcp.payload	tcp.srcport	tcp.dstport
udp.port	mqtt.msg	http.file_data

attack classes. To reduce the bias during training, Synthetic Minority Oversampling Technique (SMOTE) was used to tackle the imbalance of data to the best possible.

SMOTE performs by picking examples that are near to one another in the feature space [24]. Specifically, a random member of the minority class is first selected. Then, k closest neighbors for that case are identified (usually, $k = 5$). A neighbor is picked at random, and a synthetic example is generated at a random position between the two instances in feature space [24].

4. Methodology

In this section, the proposed approach for performing classification and detection tasks is presented in order to provide an overview of the experimental results. Deep CNN-BiLSTM has an efficient core architecture, which is used as the basis for the proposed method to demonstrate the usability and effectiveness of CNN-BiLSTM-based models. The methodology of the IDS model is presented in Fig. 5.

To compare the accuracy performance of our proposed model, we have constructed the following DL approaches alongside:

4.1. Deep neural network (DNN)

Deep neural networks (DNNs) are enhanced variants of the ordinary artificial neural network (ANN) that include numerous layers. In artificial intelligence, deep neural networks are neural networks that have a specific amount of complexity, meaning they have more than two layers [25]. Deep neural networks, which utilize advanced mathematical modeling to handle data in complicated ways, are becoming more popular [26]. The primary goal of a DNN is to take in a collection of inputs, do more sophisticated computations on them, and output results that may be used to solve real-world issues like classification and regression [27]. Fig. 6 depicted the model structure used for the DNN for our experiment.

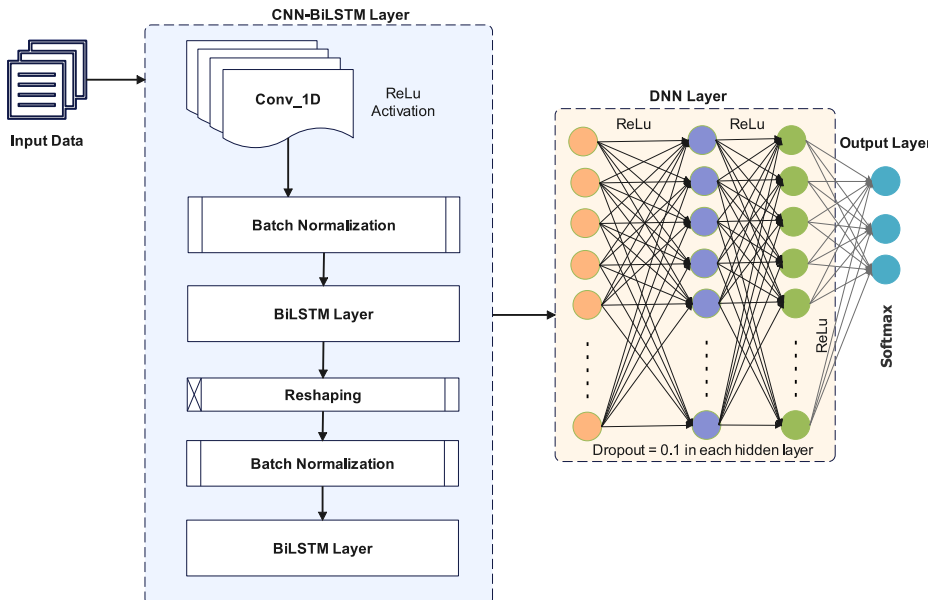


Fig. 4. Edge_IIoT Data Distribution.

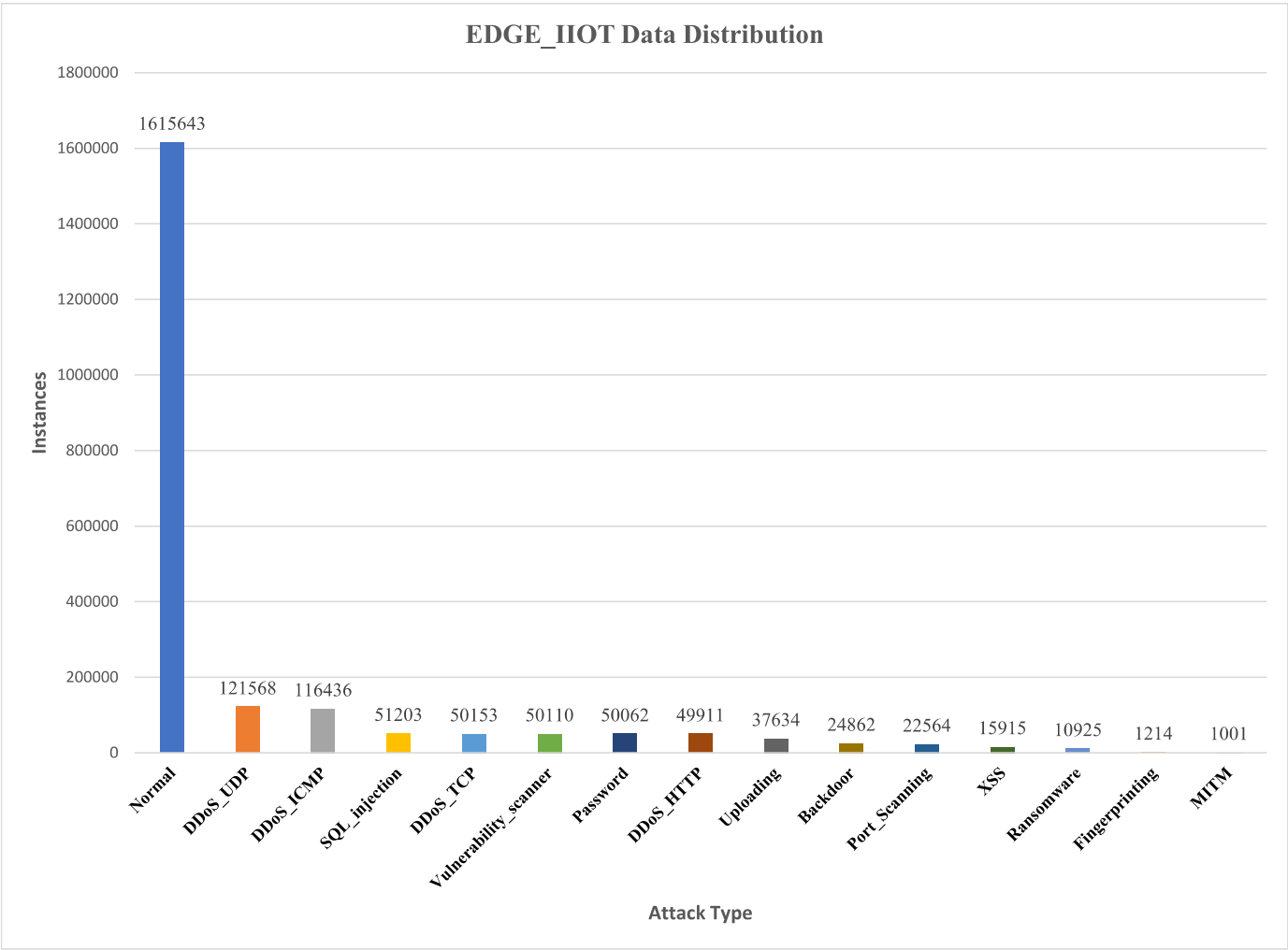


Fig. 5. Deep CNN-BiLSTM Model.

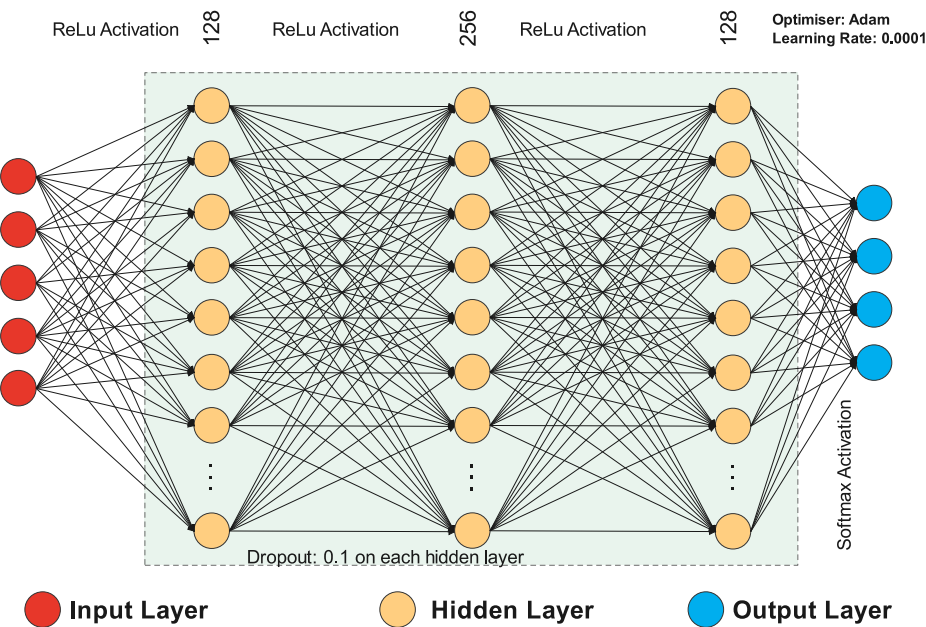


Fig. 6. DNN model.

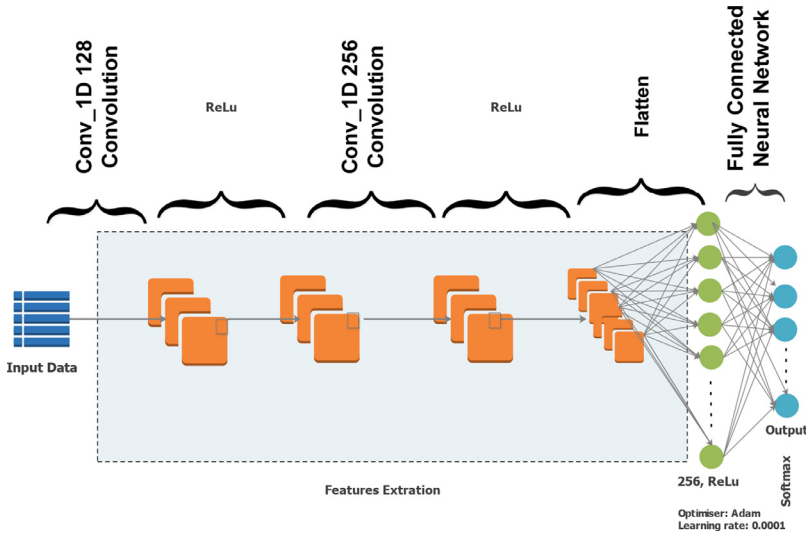


Fig. 7. CNN model.

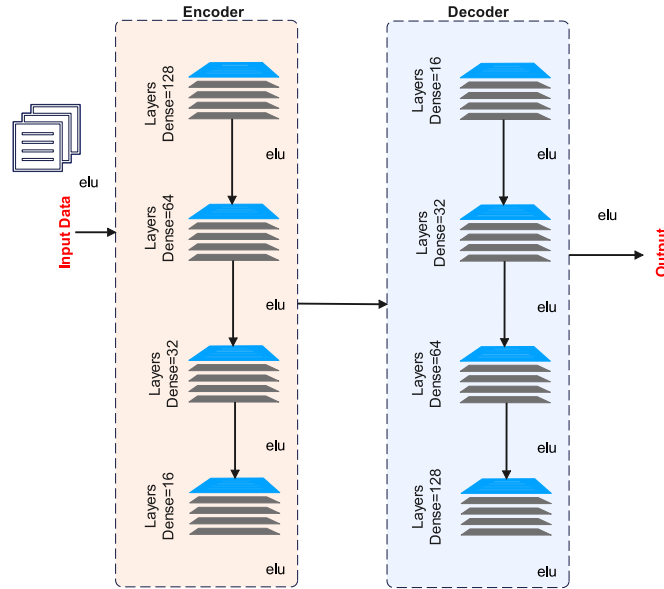


Fig. 8. AE model.

4.2. Convolutional neural network (CNN)

Multiple layers of arrays are used to process data in convolutional neural networks [28]. The objective of the convolutional layer is to extract features while preserving sequential information. For some types of challenges, such as image recognition, Convolutional Neural Networks have generated excellent results [29,30]. CNN makes use of spatial correlations present in the input data. Each successive layer of a neural network is comprised of input neuron connections. This particular area is known as the local receptive field [29]. The local receptive field concentrates on unseen neurons. The hidden neurons analyze the incoming data inside the specified field without being aware of the changes occurring outside of the border. Fig. 7 depicted the model structure used for the CNN for our experiment.

4.3. Long-Short term memory (LSTM)

Long short-term memory is an artificial neural network used in artificial intelligence and deep learning disciplines [31]. LSTM features feed-back connections, unlike conventional feedforward neural networks. Not only can this type of recurrent neural network analyze individual data points, but also whole data sequences. Hochreiter and Schmidhuber developed the LSTM algorithm. It addressed the issue of RNNs long-term dependency [32], in which the RNN is unable to predict words stored in long-term memory, but may provide more accurate predictions based

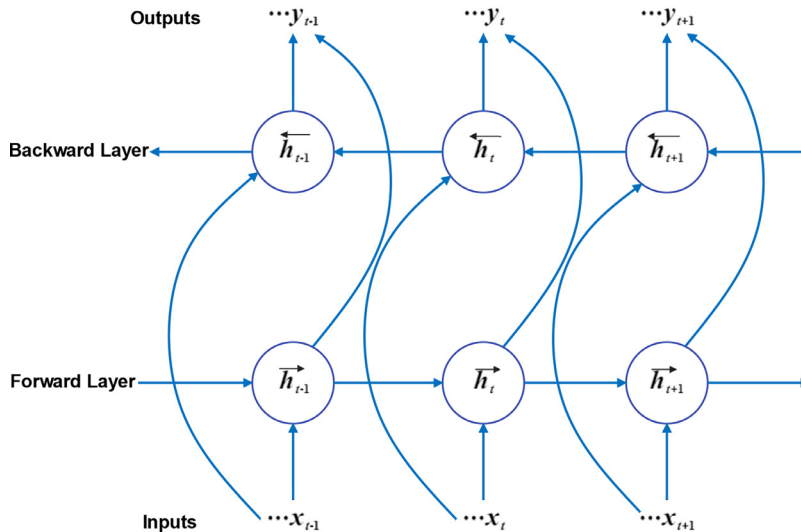


Fig. 9. Bidirectional LSTM.

Table 4
Models Performance Evaluation.

Model	Dataset	Platform	Accuracy	Loss	~Training Time (per epoch)	Inference time
DNN	CICIDS2018	With GPU Without GPU	100% 100%	0.0000 0.0000	16s 129s	9.87s 73.24s
	Edge_IIoT	With GPU Without GPU	90.82% 90.81%	0.0994 0.0995	41s 397s	23.61s 197.17s
CNN	CICIDS2018	With GPU Without GPU	100% 100%	0.0000 0.0000	27s 245s	9.43s 118.68s
	Edge_IIoT	With GPU Without GPU	92.11% 92.11%	0.0922 0.0924	49s 412s	17.25s 231.44s
AE	CICIDS2018	With GPU Without GPU	99.59% 99.58%	0.0048 0.0048	74s 750s	7.46s 74.4s
	Edge_IIoT	With GPU Without GPU	91.27% 91.27%	0.0921 0.0926	824s 11429s	80.87s 2362.94s
LSTM	CICIDS2018	With GPU Without GPU	99.96% 99.96%	0.0037 0.0037	470s 2435s	323.41s 1822.39s
	Edge_IIoT	With GPU Without CPU	94.31% 94.31%	0.0047 0.0047	738s 10700s	480.11s 7829.37s
DCNNBiLSTM	CICIDS2018	With GPU Without GPU	100% 100%	0.0000 0.0000	202s 3245s	95.04s 1712.03s
	Edge_IIoT	With GPU Without GPU	99.64% 99.62%	0.0080 0.0081	500s 8421s	219.29s 4177.53s

Table 5
Comparison of Performance with other studies.

Reference	Model	Dataset	Accuracy	Loss	Inference time
Mohammad et al. [13]	MLP	KDDCUP99 NSL-KDD ISCX-2012	90.78%	*	*
Diro and Chilamkurti [14]	DL	NSL-KDD	99.20% (B) 98.27% (M)	0.85 (B) 2.57 (M)	*
Muna et al. [15]	ADS	UNSW-NB15 NSL-KDD	98.6% (M) 92.4% (M)	1.8 8.2	2.25s 5.55s
Vinayakumar et al. [16]	C-CMU	DS1-V1 DS2-V2	99.2% 89.9%	*	*
Parra et al. [17]	DCNN LSTM	N_BaloT	94.8% (B) 94.30% (M)	*	*
Pajouh et al. [18]	RNN	Self Generated	98.18% (B)	*	*
Popoola et al. [19]	LAE-BLSTM	BotIoT	91.89% (M)	*	*
Manimurugan et al. [20]	DBN	CICIDS-2017	99.37% (M)	*	*
NG and Selvakumar [21]	VCDL	BotIoT	99.7496% (M)	*	*
Mushtaq et al. [37]	AE-LSTM	NSL-KDD	89.84% (B)	0.1100	*
This Paper	DCNNBiLSTM	CICIDS2018 Edge_IIoT	100% (M) 99.64% (M)	0.0000 0.0080	95.04s 219.29s

(B): Binary Classification (M): Multiclass Classification * : Not available

on the current input. As the gap size rises, RNNs performance becomes less efficient [32]. By default, LSTM is capable of storing information for a longer duration. Based on time-series data, it is used for processing, forecasting, and categorizing. Fig. 3 depicted the model structure used for the LSTM for our experiment.

$$f_t = \Phi(\hat{W}_f \cdot [h_{t-1}, x_t] + B_f) \quad (2)$$

$$i_t = \Phi(\hat{W}_i \cdot [h_{t-1}, x_t] + B_i) \quad (3)$$

$$\tilde{C}_t = \tanh(\hat{W}_c \cdot [h_{t-1}, x_t] + B_c) \quad (4)$$

$$C_t = f_t \times C_{t-1} = i_t \times C_t \quad (5)$$

$$O_t = \Phi(\hat{W}_o \cdot [h_{t-1}, x_t] + B_o) \quad (6)$$

$$h_t = O_t \times \tanh(\Phi(C_t)) \quad (7)$$

As shown in Fig. 3, for the Eq. 2, x_t is the network input, h_t is the output from the hidden layer, Φ denotes the sigmoid function, C_t is the cell state and the state candidate values are denoted using $\tilde{C}_t$, $\hat{W}_f$, $\hat{W}_o$, $\hat{W}_f$ and $\hat{W}_c$ are the weight used on the input, drop and output gate and memory cell, whereas B_f , B_i and B_c denote the bias for the input and output as well as the drop, gate and cell which are used in Eqs. 2, 3, 4, 5, 6 and 7. The input gate determines if input data will be retained, the drop gate determines whether data will be lost, the cell records the processing status, and the output gate delivers the result.

4.4. Autoencoder (AE)

Unsupervised artificial neural networks (ANNs) that learn how to effectively compress and encode data and then learn how to reconstruct the data back from the compressed encoded form to a representation that is as similar to the original input as feasible are known as autoencoders [33,34]. If the input characteristics were all completely independent of one another, this compression and subsequent reconstruction would be an extremely tough operation to do successfully. The goal of an

autoencoder is to learn a lower-dimensional representation (encoding) for a higher-dimensional data set, often to reduce the dimensionality of the data set being represented [35]. The AE used in this study is depicted in Fig. 8.

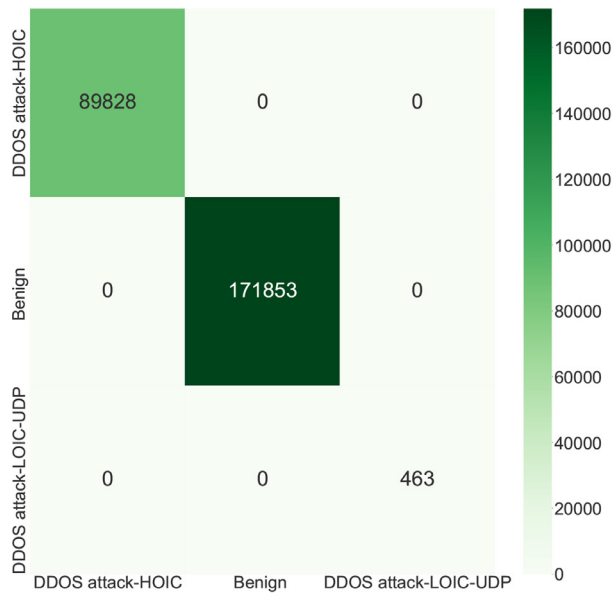
4.5. DCNNBiLSTM

The DCNNBiLSTM architecture makes use of the CNN layer for feature extraction on input data [29], as well as BiLSTMs for sequence prediction to get the desired results [36], and the DNN to boost optimizing the error and loss. The hybrid CNN and BiLSTM model was initially known as the CNN BiLSTM architecture, which stood for the Long-term Recurrent Convolutional Network, or LRCN model, and it was based on the CNN BiLSTM design. It is possible to define a Deep CNN-BiLSTM by first putting CNN layers on the front end, followed by BiLSTM layers, and finally, a DNN Layer followed by the output layer. Fig. 5 depicted the model structure used for the Deep CNN-BiLSTM and the algorithm is depicted in Algorithm 1.

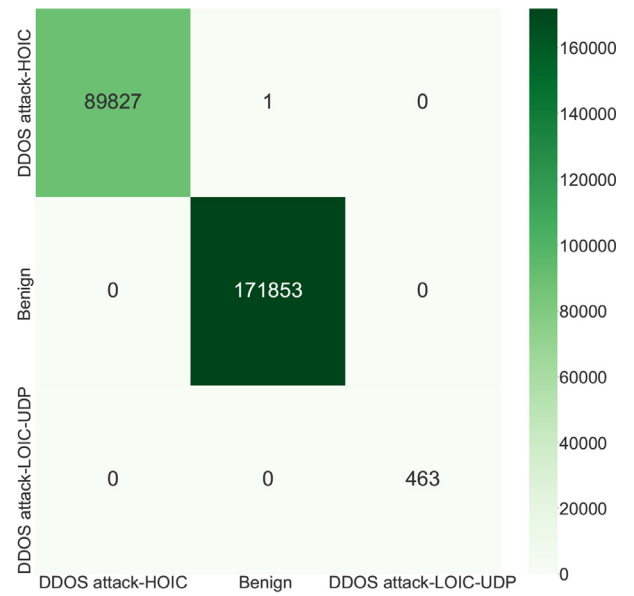
Algorithm 1 DDCNNBiLSTM model.

Require: Input df = dataset

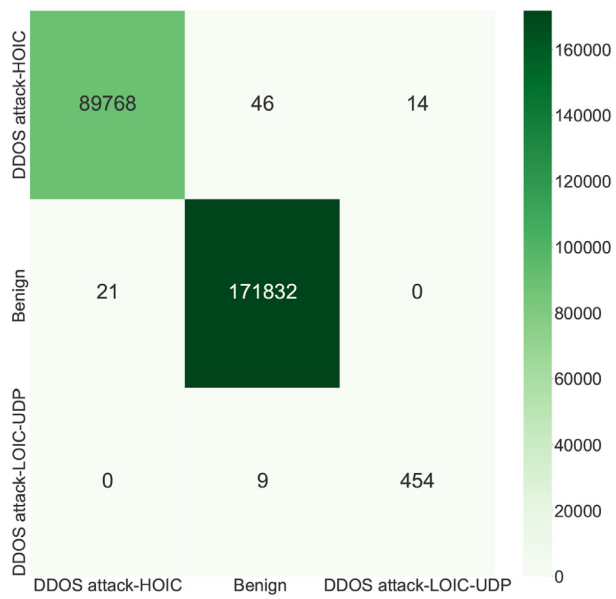
- 1: Set $Features = f_1, \dots, f_n$ from df set as Subset, where $n = no.of\ selected\ features$
- 2: Split $df_{Features}$ into training, and testing
- 3: Procedure model ()
- 4: Add Conv1D Layer to model, input shape = (76, 1), $kernelsize = n$
- 5: Set Activation functions: ReLU
- 6: Add BatchNormalisation to model
- 7: Add Bidirectional LSTM Layer to model, $Neuron = 64$
- 8: Add Reshape Layer, $input_shape = 128$
- 9: Add BatchNormalisation to model
- 10: Add Bidirectional LSTM Layer to model, $Neuron = 128$
- 11: Add Dropout Layer
- 12: Add DNN Layer o model, $Neuron = 64, 32, 16$
- 13: Set Activation functions: ReLU
- 14: Add Output Layer with $Dense = 3$
- 15: Set Activation functions: Softmax



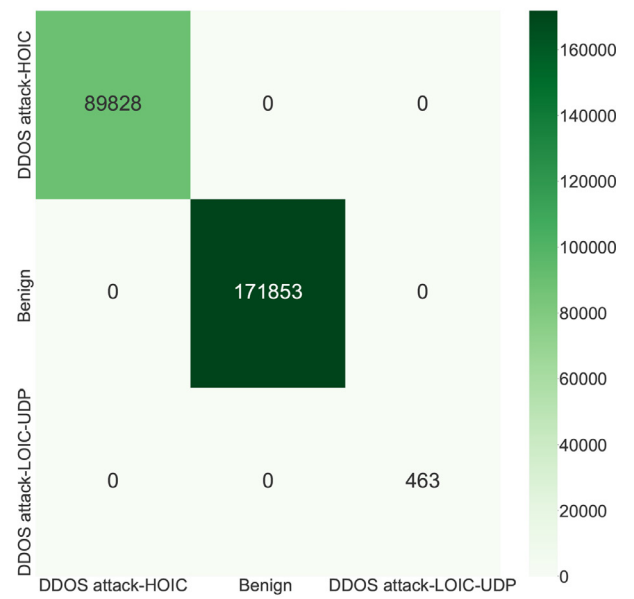
(a) DNN Confusion Matrix



(b) CNN Confusion Matrix

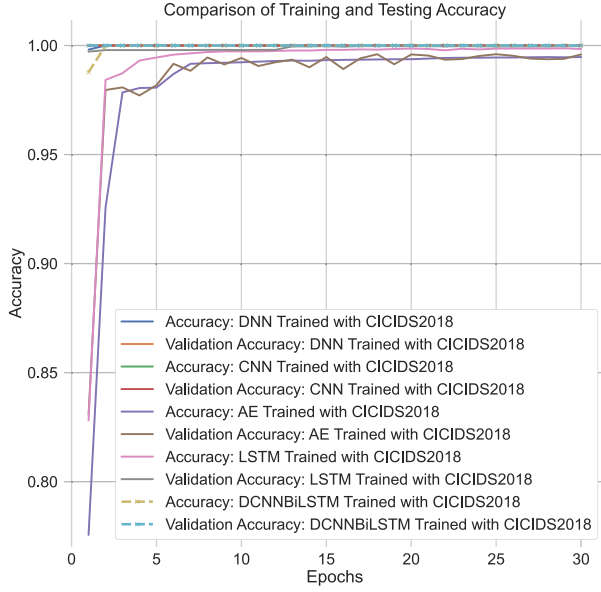


(c) LSTM Confusion Matrix

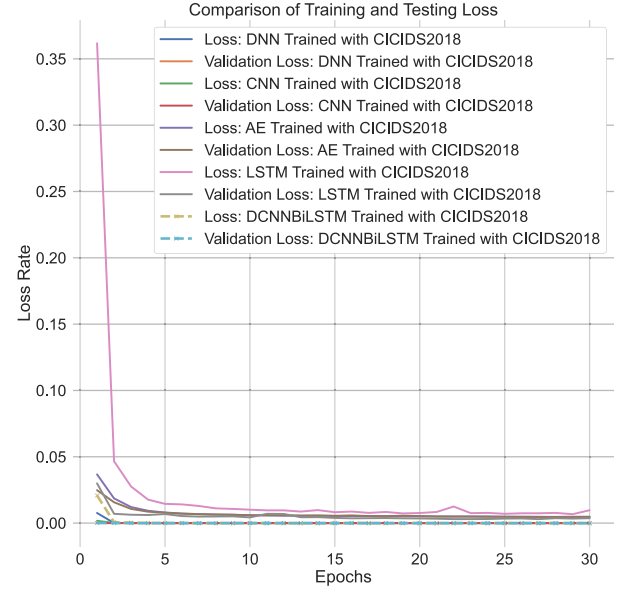


(d) Deep CNN-BiLSTM Confusion Matrix

Fig. 10. Confusion Matrix: Models trained using CICIDS2018.

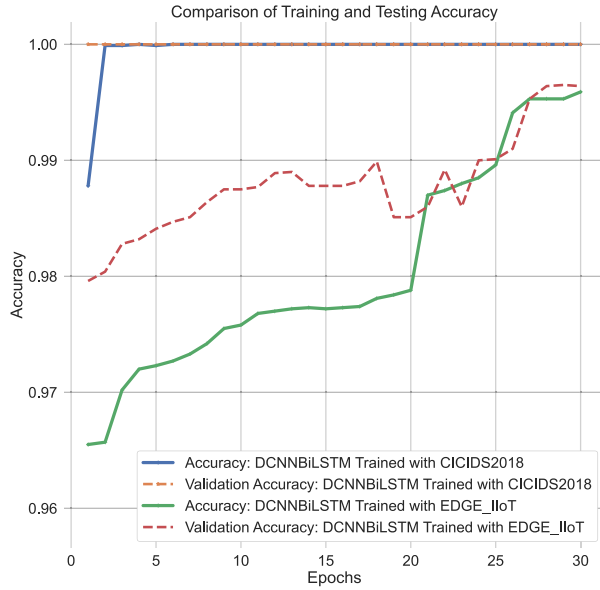


(a) Comparison of Accuracy Performance

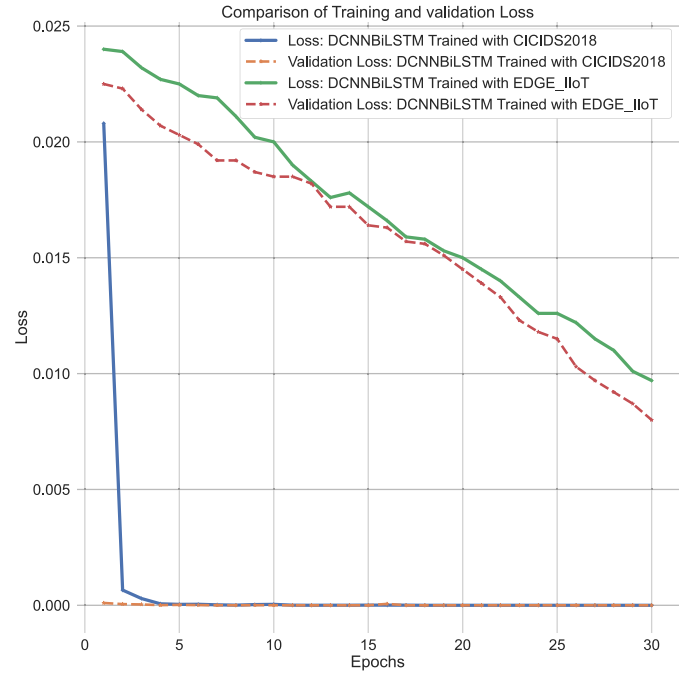


(b) Comparison of Loss Rate

Fig. 11. GPU: Models Performance trained with CICIDS2018.



(a) Comparison of Accuracy Performance



(b) Comparison of Loss Rate

Fig. 12. Models Performance trained with Edge_IIoT and CICIDS2018.

The CNN layer is activated with the ReLU function, the equation is given in Eq. 8. The Batch Normalization is formulated with Eq. 9. The softmax function is applied to the output layer with Eq. 10.

$$\text{Relu}(x) = \max(0, x) \quad (8)$$

$$\text{BatchNormalization}(x) = \frac{X - \text{Min}}{\text{Max} - \text{Min}} \quad (9)$$

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)} \quad (10)$$

The Bidirectional LSTM can be depicted as Fig. 9. Bidirectional LSTM is essentially the combination of two independent LSTMs. This framework enables the networks to have both backward and forward sequence information at each input sequence.

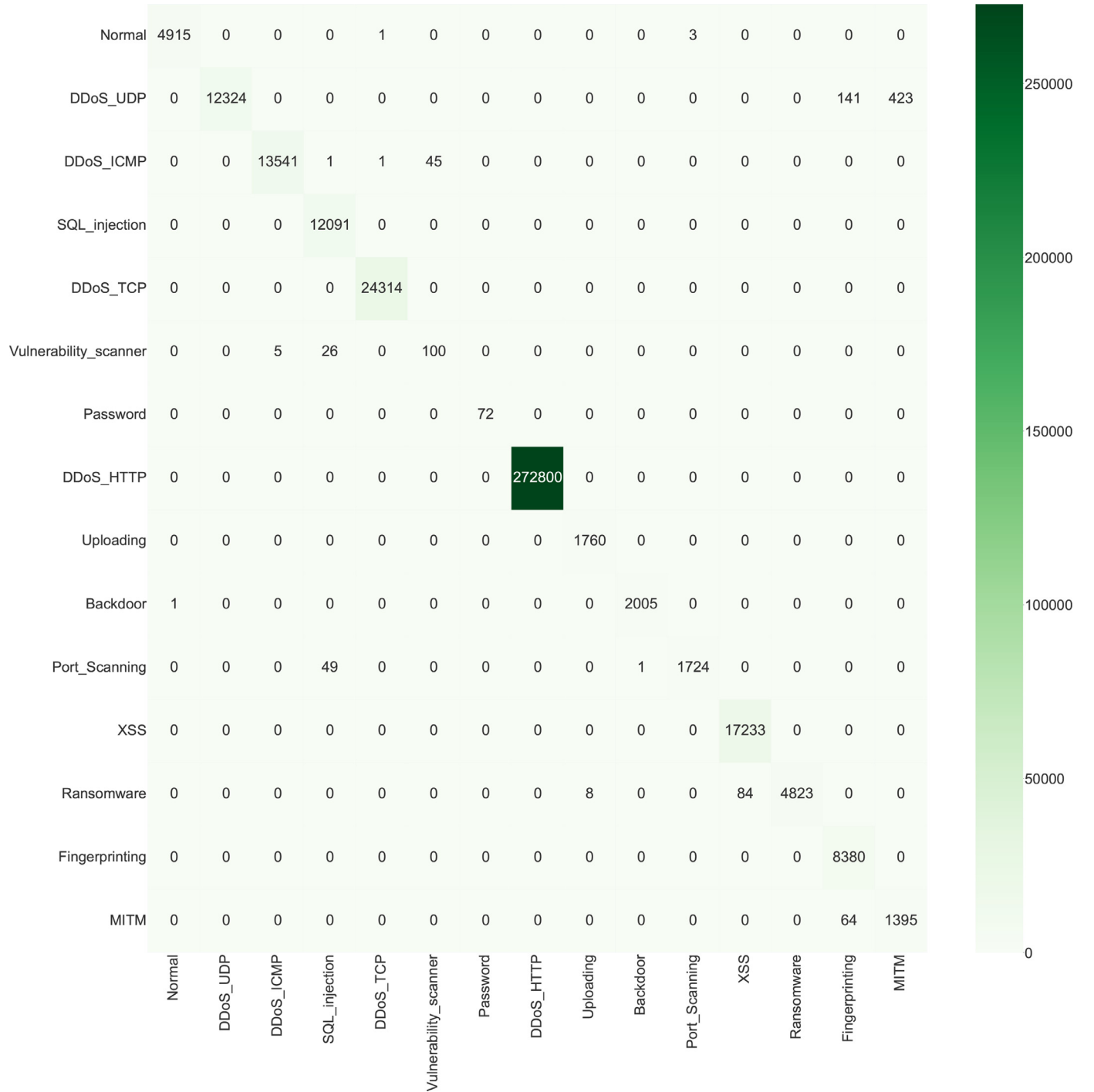


Fig. 13. DCNNBiLSTM Trained with Edge_IIoT: Confusion Matrix.

The DCNNBiLSTM neural network has been modeled with the Relu activation function, Adam optimization function, and 0.0001 learning rate settings. In addition, the loss function loss used by different numerical methods is different. The numerical method in this paper is one-hot. So use Categorical Crossentropy as the loss function.

5. Result and discussion

This paper proposes an efficient method that can be used as a modern intrusion detection system and is critical for identifying unlawful

activity via computer networks. Although several state-of-the-art algorithms have been used to classify numerous intrusion problems, their overall performance remains low, making these IDS designs challenging to implement in actual network settings.

We combine the benefits of the CNN model in extracting distinctive features and the Bidirectional LSTM models ability to capture long and short-term relationships, then add DNN with the Output Layer into a single hybrid Deep CNN-BiLSTM model. The results of the experiments demonstrated that our methodology produces effective outcomes. Experiments were performed so that a fair comparison could be made be-

tween the proposed model and current, conventional models. The outcomes of our experiments are shown in Table 4. The performance of DNN, CNN, AE, LSTM, and DCNNBiLSTM (proposed) models are compared in terms of Accuracy performance and loss rate. The model was trained and tested on GPU technology to show that the training time decreased which is shown in Table 4. The performance result has been depicted in Fig. 11 which is trained and tested using the CICIDS2018 dataset and Fig. 12 which is trained and tested using the Edge_IIoT dataset.

Fig. 10 depicted the Confusion Matrix of the DNN model (Fig. 10a), CNN model (Fig. 10b), LSTM model (Fig. 10c) and the Deep CNN-BiLSTM model (Fig. 10d). The DCNNBiLSTM is providing 100% detection accuracy rate and the loss rate is very minimal (i.e. 0.000000) as depicted in Fig. 11, whereas Fig. 12 depicted the performance of DCNNBiLSTM trained and tested using Edge_IIoT. Fig. 13 depicted the Confusion Matrix of the DCNNBiLSTM model, trained and tested using the Edge_IIoT dataset. The testing provided a 99.64% detection accuracy rate and the loss rate is very minimal (i.e. 0.0080), therefore, they are applicable for detecting DDoS attacks detection.

From Fig. 12, it appears that model trained with balance data may have better performance, whereas imbalance is having some drawbacks.

The performance of deep learning algorithms is dependent on several parameters, which may include the capacity and size of the dataset, the selected features, and the learning rate and activation. Developing the classifier based on deep learning remains an interesting field of study, and it is unreasonable to assume that a particular model is suitable for every dataset.

The primary contribution of this study is combining the CNN model with the bidirectional LSTM model which is then connected to a fully connected DNN. Existing research takes into account the fact that CNN can only extract information and needs many layers to have long-term and short-term dependencies [38], while LSTM process the sequence of input based on the structure, and it can learn long-term and short-term dependencies, and improve the models robustness, so they were combined. According to the findings of the proposed approach's experiment, the combinational model outperforms all convention models of modern IDS.

6. Conclusion

In this study, a dependable and efficient DCNNBiLSTM model for network intrusion detection based on a realistic network traffic dataset is proposed. For the purpose of ensuring the proficiency of the proposed model, some DL models have been compared in terms of detection accuracy scenarios. In terms of accuracy, the DCNNBiLSTM architecture has improved as a result of the combination of the considerable strengths of each model. In addition, it accurately detected irregular attack patterns that cannot be accurately predicted by standard machine learning techniques. The experimental results demonstrate that the proposed architecture provides more accurate predictions than the single models. The DCNNBiLSTM topology combines the benefits of CNN and LSTM. This study shows that hybrid architecture outperforms single models in the vast majority of situations. The suggested DCNNBiLSTM architecture is optimal for system administrators and network industries in order to improve IDS detection. However, even though the improved DCNNBiLSTM performs incredibly well when using a newer dataset for the training and testing datasets, there are still certain areas for improvement. DCNNBiLSTM may be used to detect zero-day attacks. Nonetheless, further research must be undertaken to investigate attack behavioral patterns and utilize the acquired data to enhance forecasting and prevention models.

Due to its complexity, the suggested model has a longer training period than typical DL approaches such as a single model, despite its advantages. This study may be expanded in the future by using a different optimization and feature selection strategy and deploying the model in real-time to categorize network traffic. We aim to imple-

ment a categorization of incoming traffic as either normal or attack type.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Appendix A. Declarations

- Funding: There is no funding support.
- Conflict of interest: The authors declare no conflict of interest.
- Ethics approval: Not Applicable.
- Consent to participate: The authors voluntarily agree to take part in this study.
- Consent for publication: All the authors read and accept to publish the manuscript.
- Availability of data and materials: Publicly available IDS dataset was used in this study. CICIDS2018 dataset can be downloaded from <https://www.unb.ca/cic/datasets/ids-2018.html>, and EdgeIIoT data can be downloaded from <https://iee-dataport.org/documents/edge-iiotset-new-comprehensive-realistic-cyber-security-dataset-iiot-and-iiot-applications>.
- Authors' contributions: "Conceptualization, V.H. and J.H.; methodology, V.H. and J.H.; software, V.H.; validation, V.H. and J.H.; formal analysis, V.H.; investigation, V.H.; resources, V.H.; data curation, V.H.; writing—original draft preparation, V.H.; writing—review and editing, V.H.; visualization, V.H.; supervision, J.H.; project administration, J.H."

References

- [1] H.-J. Liao, C.-H.R. Lin, Y.-C. Lin, K.Y. Tung, Intrusion detection system: a comprehensive review, *J. Netw. Comput. Appl.* 36 (1) (2013) 16–24, doi:[10.1016/j.jnca.2012.09.004](https://doi.org/10.1016/j.jnca.2012.09.004).
- [2] M. Al-Hawawreh, E. Sitnikova, Developing a security testbed for industrial internet of things, *IEEE Internet Things J.* 8 (7) (2020) 5558–5573, doi:[10.1109/JIOT.2020.3032093](https://doi.org/10.1109/JIOT.2020.3032093).
- [3] G. Wang, J. Hao, J. Ma, L. Huang, A new approach to intrusion detection using artificial neural networks and fuzzy clustering, *Expert Syst. Appl.* 37 (9) (2010) 6225–6232, doi:[10.1016/j.eswa.2010.02.102](https://doi.org/10.1016/j.eswa.2010.02.102).
- [4] N. Sultana, N. Chilamkurti, W. Peng, R. Alhadad, Survey on sdn based network intrusion detection system using machine learning approaches, *Peer-to-Peer Network. Appl.* 12 (2019) 493–501, doi:[10.1007/s12083-017-0630-0](https://doi.org/10.1007/s12083-017-0630-0).
- [5] F. Louati, F.B. Ktata, A deep learning-based multi-agent system for intrusion detection, *SN Appl. Sci.* 2 (4) (2020) 675, doi:[10.1007/s42452-020-2414-z](https://doi.org/10.1007/s42452-020-2414-z).
- [6] Y. Wang, Z. Ren, Y. Kong, F. Bao, Q. Dai, A hierarchical fused fuzzy deep neural network for data classification, *IEEE Trans. Fuzzy Syst.* 25 (4) (2016) 1006–1012, doi:[10.1109/TFUZZ.2016.2574915](https://doi.org/10.1109/TFUZZ.2016.2574915).
- [7] R. Zhao, R. Yan, Z. Chen, K. Mao, P. Wang, R.X. Gao, Deep learning and its applications to machine health monitoring, *Mech. Syst. Signal Process.* 115 (2019) 213–237, doi:[10.1016/j.ymssp.2018.05.050](https://doi.org/10.1016/j.ymssp.2018.05.050).
- [8] J. Yin, W. Zhao, Fault diagnosis network design for vehicle on-board equipments of high-speed railway: a deep learning approach, *Eng. Appl. Artif. Intell.* 56 (2016) 250–259, doi:[10.1016/j.engappai.2016.10.002](https://doi.org/10.1016/j.engappai.2016.10.002).
- [9] J. Wang, Y. Ma, L. Zhang, R.X. Gao, D. Wu, Deep learning for smart manufacturing: methods and applications, *J. Manuf. Syst.* 48 (2018) 144–156, doi:[10.1016/j.jmsy.2018.01.003](https://doi.org/10.1016/j.jmsy.2018.01.003).
- [10] M.A. Al-Garadi, A. Mohamed, A.K. Al-Ali, X. Du, I. Ali, M. Guizani, A survey of machine and deep learning methods for internet of things (iot) security, *IEEE Commun. Surv. Tutor.* 22 (3) (2020) 1646–1685, doi:[10.1109/COMST.2020.2988293](https://doi.org/10.1109/COMST.2020.2988293).
- [11] J. Hussain, V. Hnamte, Deep learning based intrusion detection system: modern approach, 2021 2nd Global Conference for Advancement in Technology (GCAT) (2021) 1–6, doi:[10.1109/GCAT52182.2021.9587719](https://doi.org/10.1109/GCAT52182.2021.9587719).
- [12] J. Tian, M. Gao, F. Zhang, Network intrusion detection method based on radial basic function neural network, 2009 International Conference on E-Business and Information System Security (2009) 1–4, doi:[10.1109/EBISS.2009.5138016](https://doi.org/10.1109/EBISS.2009.5138016).
- [13] M.R. Norouzian, S. Merati, Classifying attacks in a network intrusion detection system based on artificial neural networks, 13th International Conference on Advanced Communication Technology (ICACT2011) (2011) 868–873. Available from: <https://ieeexplore.ieee.org/document/5745947>

- [14] A.A. Diro, N. Chilamkurti, Distributed attack detection scheme using deep learning approach for internet of things, *Future Generat. Comput. Syst.* 82 (2018) 761–768, doi:[10.1016/j.future.2017.08.043](https://doi.org/10.1016/j.future.2017.08.043).
- [15] A.-H. Muna, N. Moustafa, E. Sitnikova, Identification of malicious activities in industrial internet of things based on deep learning models, *J. Inf. Secur. Appl.* 41 (2018) 1–11, doi:[10.1016/j.jisa.2018.05.002](https://doi.org/10.1016/j.jisa.2018.05.002).
- [16] R. Vinayakumar, M. Alazab, S. Srinivasan, Q.-V. Pham, S.K. Padannayil, K. Simran, A visualized botnet detection system based deep learning for the internet of things networks of smart cities, *IEEE Trans. Ind. Appl.* 56 (4) (2020) 4436–4456, doi:[10.1109/TIA.2020.2971952](https://doi.org/10.1109/TIA.2020.2971952).
- [17] G.D.L.T. Parra, P. Rad, K.-K.R. Choo, N. Beebe, Detecting internet of things attacks using distributed deep learning, *J. Netw. Comput. Appl.* 163 (2020) 102662, doi:[10.1016/j.jnca.2020.102662](https://doi.org/10.1016/j.jnca.2020.102662).
- [18] H. Haddadpajouh, A. Dehghantanha, R. Khayami, K.K.R. Choo, A deep recurrent neural network based approach for internet of things malware threat hunting, *Future Generat. Comput. Syst.* 85 (2018) 88–96, doi:[10.1016/j.future.2018.03.007](https://doi.org/10.1016/j.future.2018.03.007).
- [19] S.I. Popoola, B. Adebisi, M. Hammoudeh, G. Gui, H. Gacanin, Hybrid deep learning for botnet attack detection in the internet-of-things networks, *IEEE Internet Things J.* 8 (6) (2020) 4944–4956, doi:[10.1109/JIOT.2020.3034156](https://doi.org/10.1109/JIOT.2020.3034156).
- [20] S. Manimurugan, S. Al-Mutairi, M.M. Aborokbah, N. Chilamkurti, S. Ganesan, R. Patan, Effective attack detection in internet of medical things smart environment using a deep belief neural network, *IEEE Access* 8 (2020) 77396–77404, doi:[10.1109/ACCESS.2020.2986013](https://doi.org/10.1109/ACCESS.2020.2986013).
- [21] B.A. Ng, S. Selvakumar, Anomaly detection framework for internet of things traffic using vector convolutional deep learning approach in fog environment, *Future Generat. Comput. Syst.* 113 (2020) 255–265, doi:[10.1016/j.future.2020.07.020](https://doi.org/10.1016/j.future.2020.07.020).
- [22] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, *ICISSp* 1 (2018) 108–116, doi:[10.5220/0006639801080116](https://doi.org/10.5220/0006639801080116).
- [23] M.A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, H. Janicke, Edge-iiotset: a new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning, *IEEE Access* 10 (2022) 40281–40306, doi:[10.1109/ACCESS.2022.3165809](https://doi.org/10.1109/ACCESS.2022.3165809).
- [24] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, Smote: synthetic minority over-sampling technique, *J. Artif. Intell. Res.* 16 (2002) 321–357, doi:[10.1613/jair.953](https://doi.org/10.1613/jair.953).
- [25] W. Samek, A. Binder, G. Montavon, S. Lapuschkin, K.R. Müller, Evaluating the visualization of what a deep neural network has learned, *IEEE Trans. Neural Netw. Learn. Syst.* 28 (11) (2017) 2660–2673, doi:[10.1109/TNNLS.2016.2599820](https://doi.org/10.1109/TNNLS.2016.2599820).
- [26] W. Liu, Z. Wang, X. Liu, N. Zeng, Y. Liu, F.E. Alsaadi, A survey of deep neural network architectures and their applications, *Neurocomputing* 234 (2017) 11–26, doi:[10.1016/j.neucom.2016.12.038](https://doi.org/10.1016/j.neucom.2016.12.038).
- [27] L. Deng, G. Hinton, B. Kingsbury, New types of deep neural network learning for speech recognition and related applications: an overview, 2013 IEEE International Conference on Acoustics, Speech and Signal Processing (2013) 8599–8603, doi:[10.1109/ICASSP.2013.6639344](https://doi.org/10.1109/ICASSP.2013.6639344).
- [28] S. Albawi, T.A. Mohammed, S. Al-Zawi, Understanding of a convolutional neural network, 2017 International Conference on Engineering and Technology (ICET) (2017) 1–6, doi:[10.1109/ICEngTechnol.2017.8308186](https://doi.org/10.1109/ICEngTechnol.2017.8308186).
- [29] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai, et al., Recent advances in convolutional neural networks, *Pattern Recognit.* 77 (2018) 354–377, doi:[10.1016/j.patcog.2017.10.013](https://doi.org/10.1016/j.patcog.2017.10.013).
- [30] P. Kim, *Convolutional neural network*, Apress, Berkeley, CA, 2017. 121–147. doi:[10.1007/978-1-4842-2845-6_6](https://doi.org/10.1007/978-1-4842-2845-6_6).
- [31] Y. Yu, X. Si, C. Hu, J. Zhang, A review of recurrent neural networks: lstm cells and network architectures, *Neural Comput.* 31 (7) (2019) 1235–1270, doi:[10.1162/neco_a.01199](https://doi.org/10.1162/neco_a.01199).
- [32] A. Sherstinsky, Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network, *Physica D* 404 (2020) 132306, doi:[10.1016/j.physd.2019.132306](https://doi.org/10.1016/j.physd.2019.132306).
- [33] M. Tschannen, O. Bachem, M. Lucic, Recent advances in autoencoder-based representation learning, Third workshop Bayesian Deep Learn. (NeurIPS 2018) (2018), doi:[10.48550/arXiv.1812.05069](https://doi.org/10.48550/arXiv.1812.05069).
- [34] Q. Meng, D. Catchpoole, D. Skillicorn, P.J. Kennedy, Relational autoencoder for feature extraction, 2017 International Joint Conference on Neural Networks (IJCNN) (2017) 364–371, doi:[10.1109/IJCNN.2017.7965877](https://doi.org/10.1109/IJCNN.2017.7965877).
- [35] Z. Chen, C.K. Yeo, B.S. Lee, C.T. Lau, Autoencoder-based network anomaly detection, 2018 Wireless Telecommunications Symposium (WTS) (2018) 1–5, doi:[10.1109/WTS.2018.8363930](https://doi.org/10.1109/WTS.2018.8363930).
- [36] Z. Huang, W. Xu, K. Yu, Bidirectional lstm-crf models for sequence tagging (2015), doi:[10.48550/arXiv.1508.01991](https://doi.org/10.48550/arXiv.1508.01991).
- [37] E. Mushtaq, A. Zameer, M. Umer, A.A. Abbasi, A two-stage intrusion detection system with auto-encoder and lsm, *Appl. Soft. Comput.* 121 (2022) 108768, doi:[10.1016/j.asoc.2022.108768](https://doi.org/10.1016/j.asoc.2022.108768).
- [38] H. Sadr, M.M. Pedram, M. Teshnehlab, A robust sentiment analysis method based on sequential combination of convolutional and recursive neural networks, *Neural Process. Lett.* 50 (3) (2019) 2745–2761, doi:[10.1007/s11063-019-10049-1](https://doi.org/10.1007/s11063-019-10049-1).